Workshop 1: Coffee Shop Design

Class Design: Classes, Properties, Methods, Relationships, Multiplicity and Inheritance

1. Classes, Properties and Methods

Class	Properties (private)	Methods (public)
Customer	customerId: String<br/>name: String<br/>phone: String	createOrder(): Order<br/>pay(order: Order, payment: Payment
Order	orderNumber: String<br/>date: Date<br/>items: List&lt;OrderItem&gt;	addDrink(drink: Drink, quantity: int): void<br/>calculateTotal():
Drink	drinkId: String<br/>name: String<br/>price: Decimal	getName(): String<br/>getPrice(): Decimal
OrderItem	drink: Drink<br/>quantity: int	getSubtotal(): Decimal
Payment (abstract)	amount: Decimal<br/>paidAt: DateTime	pay(amount: Decimal): bool
CashPayment	(inherits Payment)	pay(amount: Decimal): bool<br/>acceptCash(): bool
QRPayment	transactionId: String	pay(amount: Decimal): bool<br/>generateQR(): String

2. Relationships and Multiplicity

• Customer 1 — 0..* Order: [illegible]

• Order 1 — 1..* OrderItem: [illegible]

• OrderItem * — 1 Drink: [illegible]

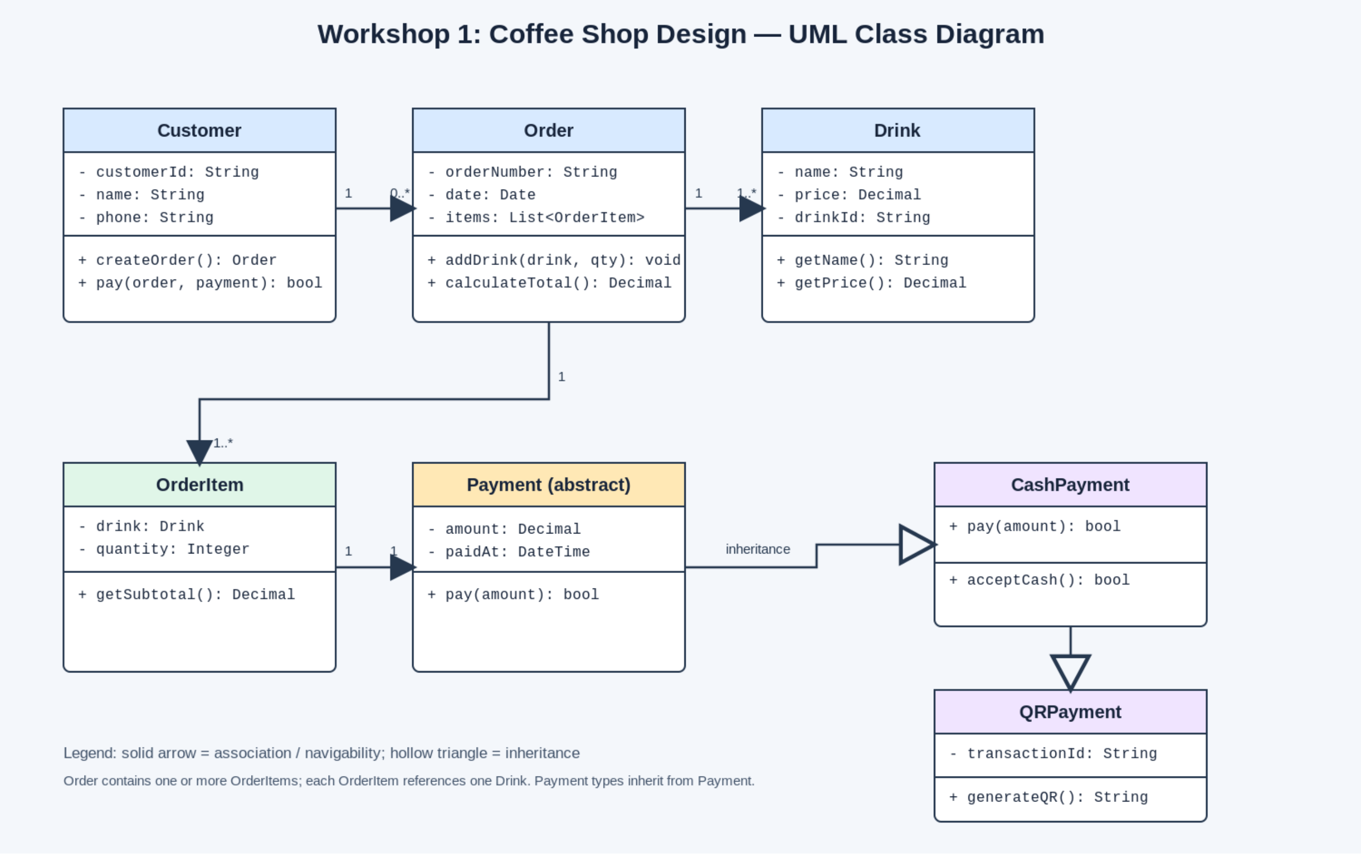
• Order 1 — 0..1 Payment: [illegible]

• CashPayment [illegible] QRPayment [illegible] (inherit) [illegible] Payment [illegible] abstract class

3. Inheritance

Payment [illegible] abstract [illegible] amount [illegible] paidAt [illegible] pay() [illegible] CashPayment [illegible] QRPayment [illegible] pay() [illegible] QRPayment [illegible] transactionId [illegible] QR code [illegible]

4. Class Diagram



Workshop 1: Coffee Shop Design — UML Class Diagram

Legend: solid arrow = association / navigability; hollow triangle = inheritance

Order contains one or more OrderItems; each OrderItem references one Drink. Payment types inherit from Payment.

Design assumptions: [illegible] Decimal; orderNumber [illegible]; calculateTotal() [illegible]; [illegible]